MOVIE RECOMMENDATION SYSTEM

Content-Based Filtering with TF-IDF and Cosine Similarity

Python Internship Project

Pandas • scikit-learn • Joblib

1. Introduction

- Recommendation systems help users discover relevant content.
- This project uses content-based filtering: similar movies are recommended based on movie information.
- It is designed as a small, transparent academic project that can run offline.
- No large user-history database is required.

2. Problem Statement & Objectives

- Problem: a user knows one movie they like but needs suggestions for what to watch next.
- Build a program that accepts a movie title and returns similar movies.
- Use genre, keywords and description as movie features.
- Rank recommendations using cosine similarity.
- Provide a simple interactive terminal application.

3. System Workflow

- Features are built by combining genre + keywords + description.
- The selected movie is compared with every other movie.

4. Key Technique: TF-IDF

- TF-IDF means Term Frequency–Inverse Document Frequency.
- It converts text into numerical feature vectors.
- Words that are useful for distinguishing movies receive more importance.
- The project also uses unigrams and bigrams (1–2 word phrases).

5. Key Technique: Cosine Similarity

- Cosine similarity compares two TF-IDF vectors.
- A larger value means the movie descriptions/features are more similar.
- The system sorts all candidates by similarity and selects the top five.
- Similarity is a text-feature score, not a guarantee that a user will like the movie.

6. Implementation

- Language: Python
- Pandas: CSV loading and data preparation
- scikit-learn: TF-IDF and cosine similarity
- Joblib: saving processed model artifacts
- Interactive terminal input: title → recommendations

7. Sample Results

Bundled academic dataset: 40 movies

Input	Top Recommendation	Similarity
Inception	Interstellar	7.67%
The Dark Knight	John Wick	7.96%
Toy Story	The Intouchables	10.71%

8. Project Structure & Execution

- code/movie_recommender.py — main interactive application
- code/train_model.py — builds and saves the recommendation artifacts
- data/movies.csv — bundled movie dataset
- artifacts/ — saved TF-IDF vectorizer and matrix
- README.md, report and presentation — documentation

```
pip install -r requirements.txt
python code/train_model.py
python code/movie_recommender.py
```

9. Limitations & Future Scope

- Limitations: small demo dataset, no real user history, recommendations depend on text quality.
- Future: larger real-world dataset, user ratings, collaborative filtering, hybrid models.
- Future UI: Streamlit or Flask web application.
- Future evaluation: Precision@K, Recall@K and user feedback.

10. Conclusion

- A complete beginner-friendly movie recommender was implemented.
- TF-IDF transforms movie text into numerical features.
- Cosine similarity ranks movies with similar content.
- The project is offline, runnable, explainable and suitable for internship demonstration.

Viva: Quick Questions

- What technique? → Content-based filtering.
- Why TF-IDF? → To convert text to weighted numerical features.
- Why cosine similarity? → To compare feature vectors.
- What is input? → Movie title.
- What is output? → Five similar movies with similarity scores.
- Production ready? → No; this is an academic demo.